RAG Assignment: Baseline vs. Re-Ranking

GitHub: https://github.com/rinkeshverma27/RAG
Code: `rag_baseline_vs_reranking.ipynb` (all code and outputs)
Proof of LLM answers: `llm_prompts_and_responses.md` (exact prompt and response for every call)

Setup

I used the GDPR (88 pages) as the document and split it into 537 chunks of about 200 words. For the bi-encoder I used `BAAI/bge-base-en-v1.5`, and for the cross-encoder `BAAI/bge-reranker-base`. The LLM is `google/gemini-2.5-flash-lite` (through OpenRouter) at temperature 0. Both pipelines use the same prompt, which tells the model to answer only from the given chunks or say "Not found in context." Everything ran on CPU.

I picked the test questions by first checking where the right chunk ranked, so they show how the baseline fails, not how often.

1. Architecture comparison

A bi-encoder encodes all the chunks once, ahead of time. For a question, it only encodes the question and compares it to the chunks with cosine similarity. The question and the chunk never see each other, so each is squeezed into one vector on its own. This is fast but not very robust. A sentence that allows something and one that forbids it share most of their words, so their vectors end up close. Words like "not" or "except" only move the vector a little.

A cross-encoder reads the question and the chunk together. Every word of the question can attend to every word of the chunk, so small but important parts, like an exception or a negation, can affect the score as much as they really matter. The downside is that nothing can be precomputed, so every pair needs its own pass through the model.

2. Rank shifts and answer comparison

Case	Question (short)	Right chunk: baseline → re-ranked	Answer 1 (baseline)	Answer 2 (re-ranked)
Exception	Excused from telling individuals about a breach because of encryption?	#5 → #1	"Not found in context."	Correct exemption (Art. 34(3)(a))
Negation	Keep sending marketing emails after the customer objects?	#4 → #3	Implied yes ("legitimate interest")	"No", data must no longer be processed (Art. 21(3))
Keyword trap	What penalties can a company face?	#6 → #1	Max fine €10M / 2% (wrong)	Max fine €20M / 4% (correct)
Buried chunk	When is a DPIA mandatory?	#6 → #2	General rule only	General rule plus the 3 mandatory cases (Art. 35(3))
Exception (retention)	Refuse erasure because a law requires keeping data?	#9 → #2	"Yes", but quoted the wrong clause	"Yes", correct exception (Art. 17(3)(b))

Case	Question (short)	Right chunk: baseline → re-ranked	Answer 1 (baseline)	Answer 2 (re-ranked)
Regression	When can data be kept longer than required?	#1 → #1	More complete	Lost a useful chunk, shorter answer
Control	Deadline to notify the supervisory authority?	#1 → #1	72 hours (correct)	72 hours (correct)

In five cases the chunk with the answer was at rank 4–9, so the LLM never saw it in the baseline. Re-ranking moved it into the top 3 every time. The baseline never invented a rule, because the prompt kept it inside the context. Instead, the missing chunk showed up as:

- a refusal (exception);
- a wrong or misleading answer (negation, keyword trap);
- a missing clause (buried chunk);
- the right answer for the wrong reason (retention).

The negation case is the clearest example of the bi-encoder problem. The baseline's top chunk says direct marketing "may be regarded as a legitimate interest", which is close in wording but opposite in meaning to the actual rule.

Re-ranking is not always better. In the regression case, the baseline already had the answer, and the cross-encoder swapped out a useful recital for an irrelevant one.

3. Latency

7 queries, 10 runs each, on CPU. LLM time is not included.

Stage	Median (ms)	Mean (ms)
Pipeline 1: baseline retrieval (top 10)	56.2	57.3
Cross-encoder re-ranking (10 pairs)	1760.5	1991.1
Pipeline 2 total	1824.2	2048.4

Pipeline 2 is about 32 times slower. The baseline only encodes the short question and does one matrix multiplication, because the chunk vectors already exist. The cross-encoder has to run the full model once for each of the 10 (question, chunk) pairs.

4. Scaling question

The same reason explains why we don't run the cross-encoder on the whole collection. Its scores can't be precomputed, so every chunk would have to be processed together with the question at query time.

I measured it on all 537 chunks: 107 seconds, about 0.2 s per pair. For 100,000 chunks that would be:

100,000 × 0.2 s ≈ 20,000 s ≈ **5.5 hours for one question**

The bi-encoder has already embedded the chunks offline, so it still answers in milliseconds. That's why the two are combined: the bi-encoder narrows the collection down to 10 candidates, and the slow but accurate cross-encoder only re-ranks those 10.